UNIVERSAL CREATIVE ENGINE (UCE)

Architecture Decision Record Pack - ADR-001 to ADR-012

Version 0.1 | 2026-09-15 | Confidential Working Baseline

Status labels: Proposed, Accepted, Superseded, Rejected. These ADRs are the initial architecture baseline and may be revised only by issuing a superseding ADR.

ADR-001: OpenUSD as scene/world interoperability foundation

Field	Value
Status	Accepted
Decision	Use OpenUSD as the long-term scene/world interchange and extensible schema foundation; UCE semantic schemas augment, not replace, it.
Rationale	Mature scene composition, hierarchy, transforms, time samples, relationships, custom schemas, broad graphics ecosystem.
Consequences / Trade-offs	Requires semantic sidecar/index for embeddings/search and may be heavy for earliest V0 prototype.

Implementation constraints

- No implementation may bypass this decision without a superseding ADR.
- Interfaces must remain versioned and testable.

ADR-002: OpenTimelineIO for editorial timeline representation

Field	Value
Status	Accepted
Decision	Represent editorial sequencing/timing with OTIO or an OTIO-compatible adapter.
Rationale	Avoid inventing timeline interchange; clean separation of media references and editorial structure.
Consequences / Trade-offs	Some UCE semantic event concepts live outside OTIO and must reference it.

Implementation constraints

- No implementation may bypass this decision without a superseding ADR.
- Interfaces must remain versioned and testable.

ADR-003: Content-addressable storage for immutable assets

Field	Value
Status	Accepted
Decision	All immutable source/derived artifacts are hash-addressed; project revisions store logical references.
Rationale	Deduplication, reproducibility, cheap branching, cache reuse, integrity verification.
Consequences / Trade-offs	Requires GC/reachability system and careful

	deletion/privacy semantics.

Implementation constraints

- Canonical project references use AssetRef objects.
- CAS writes atomic; unreachable artifacts are GC-eligible only after retention rules.

ADR-004: Local-first privacy policy

Field	Value
Status	Accepted
Decision	Default project mode is PRIVATE_LOCAL; external execution requires explicit policy allowance.
Rationale	Trust, privacy, offline studio use, free-first compute model.
Consequences / Trade-offs	Some high-end capabilities unavailable locally; router must explain constraints.

Implementation constraints

- PRIVATE_LOCAL blocks third-party APIs and outbound network.
- Policy decision is logged into operation provenance.

ADR-005: UEI independent of tracker/model

Field	Value
Status	Accepted
Decision	Persistent entity identity belongs to UCE World Graph. Track IDs and masks are evidence only.
Rationale	Backends can change; cross-shot and multi-representation identity require model independence.
Consequences / Trade-offs	UEI fusion logic becomes proprietary complexity.

Implementation constraints

- No implementation may bypass this decision without a superseding ADR.
- Interfaces must remain versioned and testable.

ADR-006: Agents plan; operation APIs mutate state

Field	Value
Status	Accepted
Decision	Agents cannot directly edit project files or arbitrary state. They propose typed operations validated by policy.
Rationale	Auditability, security, undo, reproducibility, deterministic boundaries.
Consequences / Trade-offs	May feel less flexible than unrestricted agent shell access.

Implementation constraints

- All mutations are typed operations.
- Operation coordinator performs authorization and validation.
- Agent framework remains replaceable.

ADR-007: Reconstruction-first, generation-second removal

Field	Value
Status	Accepted
Decision	Recover visible/temporal evidence before synthesizing unresolved regions.
Rationale	Reduces hallucination, cost, and unintended change.
Consequences / Trade-offs	More complex pipeline than one-shot generative edit.

Implementation constraints

- No implementation may bypass this decision without a superseding ADR.
- Interfaces must remain versioned and testable.

ADR-008: Worker process isolation

Field	Value
Status	Accepted
Decision	Media/AI/plugin work executes outside the trusted desktop process.
Rationale	Crash isolation, security boundaries, future community extensions.
Consequences / Trade-offs	IPC overhead; shared memory/GPU handles required later for performance.

Implementation constraints

- Main desktop must survive worker termination.
- Workers receive project-scoped filesystem/capability access only.

ADR-009: Signed capability manifests for models/plugins

Field	Value
Status	Accepted
Decision	Models and plugins declare capabilities, runtime, permissions, hashes, licenses, and hardware needs.
Rationale	Routing, supply-chain security, licensing, discovery, reproducibility.
Consequences / Trade-offs	Requires package tooling and signature infrastructure.

Implementation constraints

- Unsafe pickle/arbitrary-code model loading is not permitted in production trust mode.

- Hashes/signatures verified before activation.

ADR-010: Testing/privacy/security are release gates

Field	Value
Status	Accepted
Decision	No major capability ships without regression, privacy, security, and recovery tests.
Rationale	Professional reliability and trust are product requirements, not post-launch work.
Consequences / Trade-offs	Slower short-term iteration; justified by project risk.

Implementation constraints

- No implementation may bypass this decision without a superseding ADR.
- Interfaces must remain versioned and testable.

ADR-011: Qt/QML desktop shell with native graphics core

Field	Value
Status	Proposed
Decision	Use Qt/QML for workstation UI while keeping media/GPU core native and modular.
Rationale	Mature desktop docking, cross-platform, graphics integration.
Consequences / Trade-offs	License/deployment review and prototype spike required before final acceptance.

Implementation constraints

- No implementation may bypass this decision without a superseding ADR.
- Interfaces must remain versioned and testable.

ADR-012: C++ + Rust + Python language split

Field	Value
Status	Proposed
Decision	C++ for graphics/media/GPU integration, Rust for safe services/sandboxing, Python for ML/research workers.
Rationale	Matches ecosystem strengths while avoiding Python in latency-critical UI/runtime paths.
Consequences / Trade-offs	Cross-language boundary complexity; validate against team skills and build tooling.

Implementation constraints

- No implementation may bypass this decision without a superseding ADR.
- Interfaces must remain versioned and testable.

ADR Review Checklist

- Does this decision affect persistent project format?
- Does it change security/privacy boundaries?
- Does it create vendor/model lock-in?
- Does it introduce a non-commercial or restrictive dependency?
- Does it change benchmark behavior or reproducibility?
- Does it require migration tooling?
- Can it be reversed without invalidating existing projects?